

DESENVOLVIMENTO DE SISTEMAS WEB I

Aula 9: Criação de Endpoints REST

Expondo a Agenda Telefônica como uma API — sem abandonar o que já existe

Preparando o terreno para o consumo com JavaScript (próxima aula)

Objetivos da aula

1

Entender o que é REST

E por que é o padrão dominante de comunicação entre sistemas hoje.

2

Diferenciar endpoints orientados a ação dos orientados a recurso

Comparando com o que já construímos nas aulas anteriores.

3

Entender as vantagens de expor uma API

Desacoplamento, múltiplos clientes, interoperabilidade.

4

Criar o CRUD completo da Agenda em formato REST

Reaproveitando 100% do ContatoDAO já existente.

Agenda da aula

1 Recap: onde paramos

2 O problema: nossos endpoints não são REST

3 O que é REST

4 Verbos HTTP com significado semântico

5 Anatomia de uma URI REST

6 Desenhando a API da Agenda

7 Por que isso importa (vantagens reais)

8 JSON, status codes e a escolha do Gson

9 Prática: construindo o ContatoRestServlet

10 Testando a API

Onde paramos

CRUD completo e persistente

Create, Read, Update e Delete funcionando de verdade contra o PostgreSQL (Aula 8).

Camada de dados isolada

ContatoDAO concentra todo o acesso ao banco — Servlets e JSPs nunca tocam SQL diretamente.

Interface em JSP + Bootstrap

index.jsp e agenda.jsp são as únicas portas de entrada do sistema, até agora.

A limitação de hoje

Nosso sistema só "fala" com navegadores que carregam páginas JSP inteiras — nada mais consegue conversar com ele.

Nossos endpoints atuais não são REST

Olhando com atenção para o que já construímos, os Controllers são organizados por AÇÃO, não por RECURSO:

```
GET    /agenda          -- listar
POST   /novo-contato    -- criar
POST   /editar-contato -- atualizar
POST   /excluir-contato -- excluir
```

O verbo da ação está no NOME da URL ("novo-contato", "excluir-contato"), não no método HTTP. Funciona bem para páginas com formulários — mas não é um padrão que outro sistema, um app mobile, ou um script JavaScript saberia "adivinhar" sem documentação específica.

O que é REST

REST (Representational State Transfer) é um estilo arquitetural para construir APIs — um conjunto de convenções sobre como usar o HTTP de forma consistente e previsível, tratando tudo como "recursos".

Orientado a recursos

Tudo é um "substantivo" identificado por uma URI — ex: um contato, uma lista de contatos.

Usa os verbos HTTP com significado

GET, POST, PUT, DELETE não são escolhidos por preferência — cada um tem um papel definido.

Stateless

Cada requisição contém tudo que o servidor precisa — sem depender de sessão entre chamadas (Aula 1).

Representações em JSON

O recurso é "representado" como texto estruturado — hoje, quase sempre em JSON.

Verbos HTTP em uma API REST

Verbo	Significado em REST	Idempotente?
GET	Buscar um recurso (não altera nada)	Sim
POST	Criar um novo recurso	Não
PUT	Atualizar (substituir) um recurso existente	Sim
DELETE	Remover um recurso	Sim

Já vimos GET e POST desde a Aula 1. PUT e DELETE completam o conjunto que uma API REST normalmente usa para representar o CRUD inteiro.

Anatomia de uma URI REST

- 1 Substantivo, não verbo**
"/contatos", nunca "/criarContato" ou "/listarContatos" — a ação já está no verbo HTTP.
- 2 Plural para coleções**
"/contatos" representa a coleção inteira; um contato específico é um item dela.
- 3 Hierarquia por ID**
"/contatos/5" é o contato de ID 5 — dentro da coleção "/contatos".
- 4 Minúsculas, sem espaços**
Convenção comum: palavras separadas por hífen quando necessário.

```
/contatos      → a coleção inteira  
/contatos/5    → o contato de id 5
```

A API REST da Agenda Telefônica

Verbo	URI	Ação	Status de sucesso
GET	/api/contatos	Listar todos os contatos	200 OK
GET	/api/contatos/{id}	Buscar um contato	200 OK
POST	/api/contatos	Criar um novo contato	201 Created
PUT	/api/contatos/{id}	Atualizar um contato	200 OK
DELETE	/api/contatos/{id}	Excluir um contato	204 No Content

Reparem: são os MESMOS 5 métodos que o ContatoDAO já tem — só muda a "porta de entrada".

Por que criar uma API importa de verdade

Desacoplamento

A lógica de negócio (DAO) fica independente de COMO os dados serão exibidos — HTML, JSON, o que vier.

Múltiplos clientes, um só back-end

A mesma API pode alimentar o site, um futuro app mobile, ou outro sistema — sem duplicar lógica.

Interoperabilidade

JSON é um formato universal — qualquer linguagem, em qualquer plataforma, sabe interpretá-lo.

Prepara o terreno para a próxima aula

Sem uma API, não existe o que consumir com JavaScript — hoje é pré-requisito direto de amanhã.

É o que o mercado espera

Praticamente toda vaga de desenvolvimento web pede familiaridade com construção de APIs REST.

JSON: a linguagem comum das APIs

JSON (JavaScript Object Notation) é um formato de texto leve para representar dados estruturados — apesar do nome, é usado por praticamente todas as linguagens, não só JavaScript.

```
{
  "id": 1,
  "nome": "Ana Beatriz Souza",
  "telefone": "(35) 99911-2233",
  "email": "ana.souza@email.com"
}
```

POR QUE JSON VENCEU

- Leve — menos texto que XML para representar a mesma coisa
- Legível por humanos — dá para ler e editar sem ferramenta especial
- Mapeia direto para objetos — um JSON quase "é" um objeto Java, JS, Python...

Status codes em uma API REST

Código	Significado	Quando usar aqui
200 OK	Sucesso genérico	GET encontrou o recurso, PUT atualizou
201 Created	Recurso criado	POST criou um novo contato
204 No Content	Sucesso sem corpo de resposta	DELETE removeu o contato
404 Not Found	Recurso não existe	GET/PUT/DELETE com id inexistente
400 Bad Request	Requisição malformada	JSON inválido ou campo obrigatório ausente

Regra geral já vista na Aula 1: 2xx sucesso, 4xx erro do cliente, 5xx erro do servidor.

Onde a API entra no projeto

VIEW (JSP)	MODEL + DAO	NOVO: API REST
<code>index.jsp, agenda.jsp...</code> Continuam existindo, do jeito que estão.	<code>Contato + ContatoDAO</code> Reaproveitados 100% — nenhuma linha muda aqui.	<code>ContatoRestServlet</code> Um Controller novo, que devolve JSON em vez de fazer forward para JSP.

O Model e o DAO não sabem, e não precisam saber, se quem os chamou foi um Servlet JSP ou um Servlet REST.

Convertendo Java ↔ JSON com Gson

Precisamos transformar um objeto Contato em texto JSON (e vice-versa). Fazer isso manualmente, concatenando strings, é trabalhoso e sujeito a erros — vamos usar uma biblioteca para isso.

```
<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.14.0</version>
</dependency>
```

POR QUE GSON, E NÃO OUTRA COISA

É uma biblioteca simples do Google, com só duas operações principais (toJson e fromJson), sem precisar aprender um framework novo (como JAX-RS/Jersey) para uma primeira API. Frameworks mais completos ficam para uma disciplina futura.

1

ContatoRestServlet — GET (listar e buscar)

src/main/java/br/edu/faculdade/controller/ContatoRestServlet.java

```
@WebServlet("/api/contatos/*")
public class ContatoRestServlet extends HttpServlet {
    private final Gson gson = new Gson();

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {
        response.setContentType("application/json;charset=UTF-8");
        String pathInfo = request.getPathInfo(); // null ou "/5"

        if (pathInfo == null || pathInfo.equals("/")) {
            response.getWriter().write(gson.toJson(ContatoDAO.listar()));
            return;
        }

        int id = Integer.parseInt(pathInfo.substring(1));
        Contato contato = ContatoDAO.buscarPorId(id);
        if (contato == null) {
            response.setStatus(HttpServletResponse.SC_NOT_FOUND);
            return;
        }
        response.getWriter().write(gson.toJson(contato));
    }
}
```

2 POST — criando um contato

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws IOException {
    Contato novo = gson.fromJson(request.getReader(), Contato.class);
    ContatoDAO.inserir(novo);

    response.setStatus(HttpServletResponse.SC_CREATED); // 201
    response.setContentType("application/json;charset=UTF-8");
    response.getWriter().write(gson.toJson(novo));
}
```

`gson.fromJson(request.getReader(), Contato.class)` lê o corpo (body) da requisição — o JSON que o cliente enviou — e já devolve um objeto `Contato` pronto, sem precisar chamar `getParameter()` campo por campo.

3

PUT e DELETE — atualizar e excluir

```
protected void doPut(HttpServletRequest request, HttpServletResponse response)
    throws IOException {
    int id = Integer.parseInt(request.getPathInfo().substring(1));
    Contato atualizado = gson.fromJson(request.getReader(), Contato.class);
    atualizado.setId(id);
    ContatoDAO.atualizar(atualizado);
    response.setStatus(HttpServletResponse.SC_OK); // 200
}
```

```
protected void doDelete(HttpServletRequest request, HttpServletResponse response) {
    int id = Integer.parseInt(request.getPathInfo().substring(1));
    ContatoDAO.excluir(id);
    response.setStatus(HttpServletResponse.SC_NO_CONTENT); // 204
}
```

Testando a API sem interface nenhuma

Como não existe tela ainda, testamos com uma ferramenta de requisições — Postman ou a extensão Thunder Client do próprio VSCode.

GET	<code>localhost:8080/agenda-telefonica/api/contatos</code>
------------	--

GET	<code>localhost:8080/agenda-telefonica/api/contatos/1</code>
------------	--

POST	<code>localhost:8080/agenda-telefonica/api/contatos</code> (corpo JSON)
-------------	---

PUT	<code>localhost:8080/agenda-telefonica/api/contatos/1</code> (corpo JSON)
------------	---

DELETE	<code>localhost:8080/agenda-telefonica/api/contatos/1</code>
---------------	--

```
{ "nome": "Marcos Silva", "telefone": "(35) 99000-1234", "email": "marcos@email.com" }
```

MARCO ATINGIDO

A Agenda agora tem uma API de verdade.

- 1 Hoje: API REST completa (/api/contatos), reaproveitando 100% do ContatoDAO
- 2 As telas JSP continuam funcionando exatamente como antes
- 3 Próxima aula: consumir essa API com JavaScript, sem recarregar a página